

SudokuMaster

Из фотографии — в решённое sudoku

Задумка проста: сфотографировать задачу из журнала, проверить распознанные числа и наблюдать, как программа заполняет поле. Готовое решение можно сохранить картинкой.



Исходный пример: поле 16×16
с печатными числами и записями ручкой.

Что будет в первой версии

Два размера: 9×9 и 16×16.

Живой поиск: видны пробные числа и исправления неверных предположений.

Компьютер и телефон: сайт на русском языке.

Результат: полное решение в виде изображения.

Фото → проверка поля → решение → PNG

Это план будущего сайта. Далее — согласованные возможности, порядок разработки и небольшой словарь технических терминов.

1. Выводы анализа

Первую версию можно построить полностью без бэкенда: распознавание фотографии, решение sudoku и создание изображения выполняются в браузере. Сервер потребуется только для раздачи файлов сайта. Tesseract.js поддерживает распознавание через WebAssembly непосредственно в браузере. [1]

Согласованные возможности:

- Классические sudoku **9×9** и **16×16**, с блоками 3×3 и 4×4.
- Распознавание печатных чисел; попытка исключить рукописные ответы и кандидаты.
- Обязательная проверка распознанного поля пользователем.
- Отображение реального поиска, включая гипотезы и откаты.
- Работа на компьютерах и телефонах; интерфейс на русском.
- Скачивание готового решения в PNG.

Главная техническая неопределённость — распознавание исписанных фотографий. На вашем снимке есть двузначные числа, наклон, неравномерное освещение и записи ручкой. Точность OCR на нём пока не измерена; автоматическое отделение любых рукописных записей обещать нельзя.

Папка SudokuV2 пустая. В соседнем файле full.py есть терминальный решатель и поле из 104 исходных чисел, соответствующее примеру. Данные используем как основу теста после сверки с фотографией; браузерный решатель реализуем отдельно.

2. Пользовательский сценарий

1. **Загрузка.** Пользователь выбирает фотографию или перетаскивает её на страницу. Поддерживаем JPEG, PNG и WebP; для неподдерживаемого формата показываем понятное сообщение. Дополнительно доступны ручной ввод и демонстрационный пример.
2. **Выделение поля.** Приложение находит границы и предполагает размер 9×9 или 16×16. Пользователь может изменить размер, повернуть изображение и поправить четыре угла. На фотографии с несколькими задачами обрабатывается одно выбранное поле.
3. **Проверка распознавания.** Показываем фотографию рядом с редактируемой сеткой; на телефоне — переключение между ними. Выделяем сомнительные клетки и конфликты. При выборе клетки показываем увеличенный фрагмент оригинала. Исправлять можно любую клетку.
4. **Решение.** После подтверждения поля пользователь нажимает «Решить». Исходные числа, логические выводы текущей ветки и гипотезы визуально различаются. Неверная ветка приводит к видимому удалению зависимых чисел.
5. **Управление.** Доступны пауза, продолжение, изменение скорости, остановка и быстрое завершение. Остановка возвращает к подтверждённой задаче для редактирования.

6. Результат. Показываем заполненное поле и статус: единственное решение, несколько решений либо единственность ещё не установлена. Кнопка «Скачать PNG» сохраняет чёткую сетку на белом фоне с различием исходных и найденных чисел.

Ошибки исходного поля подсвечиваем до запуска. Если противоречие обнаружено только при поиске, возвращаем пользователя к проверке фотографии. Распознанные числа автоматически не заменяем ради получения решения.

3. Архитектура и алгоритмы

Часть	Решение
Интерфейс	React, TypeScript, Vite, обычный CSS
Обработка фотографии	OpenCV.js
Распознавание чисел	Tesseract.js с числовым набором символов
Решатель	Отдельный модуль TypeScript
Фоновые вычисления	Web Workers
Экспорт	Canvas → PNG
Проверки	Vitest и Playwright

Распознавание. Сначала корректируем ориентацию, находим сетку и выправляем перспективу. OpenCV.js предоставляет преобразование по четырём углам. Затем выделяем клетки по линиям сетки, убираем рамки, нормализуем фон и распознаём числа по отдельным клеткам. Для 16×16 читаем целое число, включая 10–16. [2]

Цветные записи подавляем по цвету, мелкие заметки — по размеру и расположению. Это эвристики: сомнительные результаты требуют проверки. Сохраняем исходный фрагмент и уверенность OCR; нераспознанное содержимое показываем как требующее внимания. Tesseract.js позволяет ограничивать символы и распознавать отдельные области изображения. [3]

Решатель. Используем распространение ограничений и поиск с возвратом:

- Кандидаты определяются строкой, столбцом и блоком; для вычислений используются битовые маски.
- Сначала применяются одиночные кандидаты и единственные возможные позиции числа.
- Если выводов недостаточно, выбирается клетка с минимальным числом кандидатов; варианты проверяются в стабильном порядке.
- При противоречии полностью восстанавливается состояние до гипотезы.

- После первого решения поиск продолжается до второго решения либо исчерпания вариантов, чтобы проверить единственность. Найденное первое решение остаётся на экране.

Предусматриваем лимит 30 секунд активного вычисления с возможностью продолжить поиск. Ожидание анимации и пауза в лимит не входят. Достижение лимита означает незавершённый поиск, а не отсутствие решения.

Связь с интерфейсом. Решатель отправляет события установки числа, гипотезы, отката и завершения. Обработка выполняется порциями, очередь событий ограничена; медленная анимация приостанавливает производство новых событий. Быстрый режим обновляет поле пакетами. Это сохраняет отзывчивость интерфейса при большом поиске. [4]

Внутренние контракты:

- **PuzzleDefinition:** размер, геометрия блоков и подтверждённые исходные числа.
- **RecognitionResult:** предложенное поле, уверенность и признаки необходимости проверки.
- **SolverEvent:** изменения поля и состояние поиска.
- **SolveResult:** найденное решение и результат проверки единственности.

HTTP API не требуется. Правила sudoku отделяем от интерфейса и OCR, чтобы позже добавлять другие варианты.

Библиотеки, WASM и данные OCR размещаем вместе с сайтом и загружаем по необходимости. Фотографии остаются в памяти устройства; аккаунты, база данных и история загрузок в первую версию не входят.

4. Порядок реализации и проверка готовности

1. **Проверить распознавание на примере.** Сверить печатные числа фотографии с Python-данными; проверить выравнивание, удаление ручки и распознавание 1–16. Зафиксировать пропуски, ошибочные числа и количество ручных исправлений.
2. **Собрать редактируемое поле и решатель.** Сначала обеспечить полный сценарий с ручным вводом и демонстрационными задачами, затем подключить события поиска.
3. **Подключить фотографию.** Реализовать выделение границ, OCR, прогресс обработки и экран проверки. Ошибка загрузки OCR должна оставлять доступным ручной ввод.
4. **Завершить интерфейс и PNG.** Добавить мобильное увеличение поля, отдельную панель ввода чисел 1–16, управление поиском и экспорт изображения из данных решения.
5. **Провести локальную приёмку.** Запуск через `npm install` и `npm run dev`; отдельно проверить сборку через `npm run build` и `npm run preview`.

Обязательные проверки:

- Решение задач обоих размеров; сохранение исходных чисел и корректность каждой строки, столбца и блока.
- Противоречивые исходные данные, отсутствие решения, несколько решений и остановка по лимиту.
- Совпадение поля после применения всех событий с результатом решателя; правильное восстановление при откатах.
- Чистые изображения, наклон, тени, двузначные числа и предоставленная фотография с ручкой.
- Пауза, остановка и новая загрузка во время вычислений; запоздалые события предыдущей задачи не меняют новое поле.
- Полный путь «фото → исправление → решение → PNG» в настольных браузерах, Safari на iPhone и Chrome на Android.
- Совпадение PNG с решением и отсутствие отправки фотографии по сети.

Критерий готовности: пользователь может пройти весь сценарий на обоих размерах, исправить ошибки распознавания и получить корректное решение без зависания страницы.

5. Публикация и границы первой версии

Рекомендуемое размещение — Cloudflare Pages. После локальной приёмки публикуем статическую сборку dist, получаем адрес *.pages.dev, позднее подключаем домен. Для чисто статических проектов доступны бесплатные запросы; на бесплатном плане действуют ограничения, включая 500 сборок в месяц и 25 MiB на отдельный файл. Размеры файлов OpenCV/WASM проверяем перед публикацией. [5, 6]

Альтернатива — GitHub Pages, если удобнее держать публикацию рядом с репозиторием. [7]

Первая версия поддерживает сложные задачи с классическими правилами. Killer Sudoku, диагональные и нерегулярные поля, распознавание рукописных ответов, учебные объяснения и полноценный офлайн-режим остаются последующими этапами. Русские тексты выносим в отдельный словарь для будущего перевода.

Небольшой словарь

- **Браузер** — приложение, в котором открывают сайты: например, Chrome или Safari.
- **Бэкенд** — программа на удалённом сервере, которая обрабатывает запросы. В этом проекте вычисления планируется выполнять на устройстве пользователя.
- **OCR** — автоматическое распознавание текста и чисел на изображении.
- **Кандидаты** — числа, которые ещё могут находиться в определённой клетке по правилам sudoku.
- **Гипотеза и откат** — пробная постановка числа и возвращение к предыдущему состоянию, если предположение привело к противоречию.
- **WebAssembly / WASM** — технология, позволяющая запускать вычислительные модули в браузере.
- **Web Workers** — фоновые исполнители в браузере: они считают, пока интерфейс остаётся доступным.
- **PNG** — формат изображения, подходящий для чёткого сохранения сетки и чисел.

Документация и источники

[1] Tesseract.js — распознавание в браузере

[2] OpenCV.js — геометрические преобразования

[3] Tesseract.js — программный интерфейс

[4] MDN — Web Workers

[5] Cloudflare Pages — статические запросы

[6] Cloudflare Pages — ограничения

[7] GitHub Pages — ограничения

Сведения о технологиях и размещении проверены при подготовке плана 13 сентября 2026 года. Условия платформ перед публикацией следует перепроверить.